

Comandos básicos do MongoDB



Comparação Direta: Relacional vs. MongoDB

Para entender o conceito facilmente, veja como os termos se traduzem do modelo tradicional (SQL) para o MongoDB (NoSQL): [YouTube · UNIVESP](#)

Banco Relacional (SQL)	MongoDB (NoSQL)	O que significa?
Banco de Dados	Banco de Dados (Database)	O contêiner principal.
Tabela	Coleção (Collection)	O agrupamento de registros.
Linha / Registro	Documento (Document)	Um registro individual de dados.
Coluna	Campo (Field)	Uma propriedade ou chave do dado.

Entrada e saída:

```
mongosh
```

```
exit
```

Vamos criar um banco de dados para uma biblioteca, uma coleção para livros e uma coleção para membros.

Criação de um banco de dados:

O comando para se criar bancos de dados no MongoDB é `use nome_do_banco_de_dados`. Nosso banco de dados se chamará biblioteca. Então, o comando para a criação será:

```
use biblioteca
```

Ver todos os bancos de dados:

```
show databases
```

ou

```
show dbs
```

Criação de uma coleção:

Uma coleção seria o equivalente, em um banco de dados relacional, a uma tabela.

Ao usarmos db antes da função, estamos criando uma coleção no banco de dados que foi definido no comando anterior, ou seja, no banco biblioteca.

```
db.createCollection("livros")
```

E agora vamos criar a coleção de usuários, que, como definimos anteriormente, se chamará membros:

```
db.createCollection("membros")
```

Inserir um documento:

Um documento do MongoDB seria o equivalente, em um banco de dados relacional, aos registros de uma tabela. O comando para inserção de um documento seria insertOne:

```
db.nome_da_colecao.insertOne({campo1: valor1, campo2: valor2, ...})
```

Para inserir os dados de um livro em um documento, nosso comando, além de usar o db, precisa declarar a qual a coleção o documento pertence. Então o comando se inicia com db.nome_da_coleção, e só depois o insertOne é acrescentado.

```
db.livros.insertOne({titulo: "Percy Jackson e o Ladrão de Raios", autor: "Rick Riordan", editora: "Pendragon", lancamento: new Date("2023-07-12")})
```

Vamos inserir mais um livro.

```
db.livros.insertOne({titulo: "Pega-pega com Folclóricos", autor: "Rod Zandonadi", editora: "Griots", lancamento: new Date("2022-08-12")})
```

Listar documentos no MongoDB:

Agora temos dois livros cadastrados em nosso banco de dados. Para listá-los, usamos o comando find.

```
db.livros.find()
```

Inserir mais de um documento no MongoDB:

Vamos, agora, criar alguns usuários, os membros da nossa biblioteca. Porém, para não precisarmos usar o comando `insertOne` várias vezes, vamos utilizar um novo comando. O comando `insertMany`, que nos permite inserir vários documentos de uma vez.

Da mesma forma, usaremos `db` e o nome da coleção antes do comando, no caso, a coleção será a de membros.

```
db.membros.insertMany([
  { nome: "Rodrigo Estagiário", dataNasc: new Date("1991-10-28"), sexo: "M",
    escola: "Eitor Freitas" },
  { nome: "João Estagiário", dataNasc: new Date("1992-12-13"), sexo: "M", escola:
    "Possidônio" },
  { nome: "Alysson Estagiário", dataNasc: new Date("1990-11-15"), sexo: "M",
    escola: "Possidônio" },
  { nome: "Bruna Estagiária", dataNasc: new Date("1995-12-26"), sexo: "F", escola:
    "Eitor Freitas" },
  { nome: "Aymin Estagiária", dataNasc: new Date("1990-08-28"), sexo: "F", escola:
    "Eitor Freitas" },
  { nome: "Nicole Estagiária", dataNasc: new Date("2000-03-27"), sexo: "F", escola:
    "Eitor Freitas" },
])
```

Rode novamente o comando `find` para listar todos os membros. Não se esqueça de trocar a coleção para membros:

```
db.membros.find()
```

CRUD no MongoDB

Como todo bom tutorial de desenvolvimento web, é imprescindível termos um CRUD. Vamos realizar então um CRUD de livros.

CRUD de Livros

Inserir um documento:

```
db.livros.insertOne({titulo: "A Filha do Tempo e os Descendentes de Etherion",  
autor: "Rod Zandonadi", editora: "Griots", lancamento: new Date("2021-02-07")})
```

```
db.livros.insertOne({titulo: "O pior Rei Artur de todos os tempos", autor: "Rod  
Zandonadi", editora: "", lancamento: new Date("2021-02-07")})
```

Como vimos anteriormente, o comando `insertOne` insere um documento na coleção especificada no MongoDB. Você pode usar `insertMany()` para inserir vários documentos de uma vez:

```
db.livros.insertMany([  
  
  {titulo: "A Filha do Tempo e as Armas Primervas", autor: "Rod Zandonadi",  
editora: "Griots", lancamento: new Date("2022-02-07")},  
  
  {titulo: "A Filha do Tempo e os Escolhidos dos Primeiros", autor: "Rod  
Zandonadi", editora: "Griots", lancamento: new Date("2023-02-07")},  
  
  {titulo: "Por Nidavellir", autor: "Rod Zandonadi", editora: "Griots", lancamento:  
new Date("2022-02-07")},  
  
  {titulo: "Percy Jackson e o Último Olimpiano", autor: "Rick Riordan", editora:  
"Pendragon", lancamento: new Date("2023-07-12")}  
])
```

Consultar documentos no ****MongoDB:****

```
db.livros.find()
```

Já aprendemos o comando `find` retorna todos os documentos na coleção especificada.

Consultar documentos com filtro no ****MongoDB:****

```
db.nome_da_colecao.find({campo: valor})
```

Isso retorna os documentos que correspondem ao filtro especificado. Por exemplo, {editora: "Griots"} retornará os documentos com o campo editora igual a Griots.

```
db.livros.find({editora: "Griots"})
```

Podemos consultar também os autores:

```
db.livros.find({autor: "Rick Riordan"})
```

Ou qualquer outro campo que exista na coleção.

Atualizar um documento no ****MongoDB:****

```
db.nome_da_colecao.updateOne({filtro}, {$set: {campo: novo_valor}})
```

Esse comando atualiza um documento que corresponda ao filtro especificado. Um uso comum é realizar a atualização pelo ID. Você pode pegar esse ID com o comando find, que lista todos os documentos da coleção. Vamos supor que você queria listar todos os livros da editora Griots:

```
db.livros.find({editora: "Griots"})
```

Agora é só escolher o documento que você quer atualizar, e inserir o valor do _id no comando updateOne.

Para copiar o ID do terminal, selecione o ID, clique com o botão direito sobre a seleção e escolha copiar; ou, depois de selecionar o ID, aperte as teclas CTRL + SHIFT + C.

Para colar a informação no terminal, repita os passos anteriores, mas escolha colar depois de clicar com o botão direito; e use CTRL + SHIFT + V para colar.

```
db.livros.updateOne(  
  { _id: ObjectId("id_do_documento") },  
  { $set: { campo: novo_valor } }  
)
```

Use `updateMany()` para atualizar vários documentos. Vamos supor que você queira adicionar o campo ilustrador a todos os livros da editora Griots:

```
db.minha_colecao.updateMany(  
  { editora: "Griots" },  
  { $set: { ilustrador: "Rodzanart" } }  
)
```

Excluir documentos no **MongoDB:******

```
db.nome_da_colecao.deleteOne({filtro})
```

O comando `deleteOne` exclui um documento que corresponda ao filtro especificado. Vamos criar um livro com o nome errado, e excluí-lo com base no nome.

```
db.livros.insertOne({titulo: "Livro para excluir", autor: "Rod Zandonadi", editora:  
"Forgus", lancamento: new Date("2021-02-07")})
```

E agora usaremos o comando `deleteOne`.

```
db.livros.deleteOne({ titulo: "Livro para excluir" })
```

Que tal confirmarmos com o comando `find`, filtrando pela editora Forgus?

```
db.livros.find({editora: "Forgus"})
```

Vamos usar `insertMany` para criar três livros e deletá-los usando `deleteMany()`.

```
db.membros.insertMany([
  {titulo: "Livro para excluir I", autor: "Rod Zandonadi", editora: "Forgus",
  lancamento: new Date("2022-02-07")},
  {titulo: "Livro para excluir II", autor: "Rod Zandonadi", editora: "Forgus",
  lancamento: new Date("2023-02-07")},
  {titulo: "Livro para excluir III", autor: "Rod Zandonadi", editora: "Forgus",
  lancamento: new Date("2022-02-07")},
])
```

E depois

```
db.minha_colecao.deleteMany({ editora: "Forgus" })
```

Falta agora conferir se todos os livros da editora Forgus foram excluí-dos, com o comando find:

```
db.livros.find({editora: "Forgus"})
```

Contar documentos no **MongoDB:******

Esse é um bônus no nosso CRUD.

O comando countDocuments retorna o número de documentos que correspondem ao filtro especificado.

```
db.minha_colecao.countDocuments({filtro})
```

Para saber quantos livros da editora Griots temos em nosso sistema, basta usar o comando da seguinte forma:

```
db.livros.countDocuments({editora: "Griots"})
```

Esses são apenas alguns comandos básicos do MongoDB para começar. O *MongoDB* oferece uma ampla variedade de recursos e comandos avançados para manipulação de dados. Se vocês gostarem do artigo, trarei mais tutoriais para nos aprofundarmos no MongoDB.